



AHEAD OF EVERYONE

Innovating the Future, Today.

Daily Tech Digest | June 10, 2026

CURATED INSIGHTS FOR THE MODERN TECH LEADER

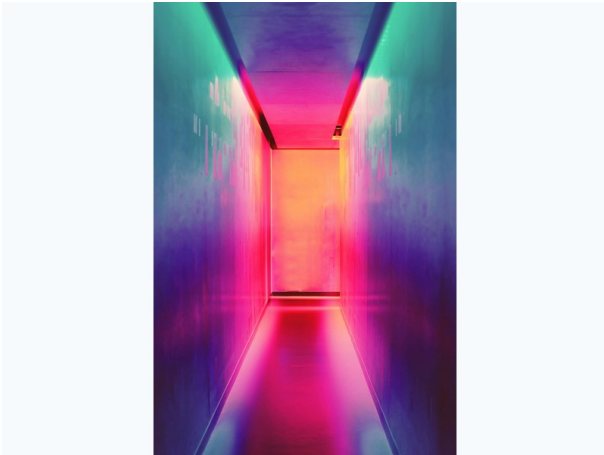
© 2026 Ahead of Everyone. All Rights Reserved.

TABLE OF CONTENTS

- 01.** Building an HTML-first site doubled our users overnight..... **Pg 3**
- 02.** AMA: I'm Eric Ries (The Lean Startup) & Author of Ne... **Pg 4**
- 03.** Apache Burr: Build reliable AI agents and applications..... **Pg 5**
- 04.** PgDog is funded and coming to a database near you **Pg 6**
- 05.** MercedesBenz starts largescale production of electri... **Pg 7**



01. Building an HTML-first site doubled our users overnight



This is a story of how building HTML-first doubled a company's users literally overnight. My client was a utility company, and they had a big problem. To apply for their services, customers could either use an old ASP form on the website, or follow a manual process. The manual process was more expensive for the company, of course. Adding a lot of pressure, this was a regulated monopoly, and if their customer satisfaction dropped below 96% (if I remember correctly) it could result in millions of pounds in fines. There were two previous failed (and very expensive) attempts to solve the problem. In the most recent, contractors in another country had built a React app. The React app was online for 3 days before being pulled because of customer complaints. I took one look at it and told my boss we can't take ownership of this. It was a mess of loading spinners and global javascript states. It was not accessible. Image upload was a vital part of the form, and it attempted to store images (along with all other form data) in localStorage which has a 5mb limit! I took a very bold decision and built a new version of

the site using Astro. It was HTML-first. Javascript existed, in web components, but only to progressively-enhance a website that worked perfectly fine without it. I was very moved by this anecdote from Terence Eden: A few years ago I was doing policy research in a housing benefits office in London. They are singularly unlovely places. The walls are brightened up with posters offering helpful services for people fleeing domestic violence. The security guards on the door are cautiously indifferent to anyone walking in. The air is filled with tense conversations between partners – drowned out by the noise of screaming kids. In the middle, a young woman sits on a hard plastic chair. She is surrounded by canvas-bags containing her worldly possessions. She doesn't look like she is in a great emotional place right now. Clutched in her hands is a games console – a PlayStation Portable. She stares at it intensely; blocking out the world with Candy Crush. Walking behind her, I glance at her console and recognise the screen she's on. She's connected to the complementary WiFi and is browsing the GOV.UK pages on Housing Benefit. She's not slicing fruit; she's arming herself with knowledge. The PSP's web browser is – charitably – pathetic. It is slow, frequently runs out of memory, and can only open 3 tabs at a time. But the GOV.UK pages are written in simple HTML. They are designed to be lightweight and will work even on rubbish browsers. They have to. This is for everyone. The basic setup ended up being that each step in the form wizard was its own page. When the ... [Content truncated. Click below to read more]

READ FULL SOURCE ARTICLE ->



02. AMA: I'm Eric Ries (The Lean Startup) & Author of New Bestseller Incorruptible



It's been fifteen years since I wrote *The Lean Startup*, and in that time I've seen some things. In both big companies and tiny startups, NGOs and governments, in almost every industry you can name. I've helped a lot of people create a lot of amazing companies, but I've also seen so many ways this can go wrong. There's a darkness in our industry that we often don't talk about. I kept watching good companies drift away from the missions they were founded on. Not because anyone woke up one day and decided to be evil, but because the structure they were built on slowly pulled them there. I call that pull "financial gravity." We've all experienced watching a company we love or admire be warped and broken beyond recognition; until it's a husk of its former self, or worse. I wanted to understand why. And I wanted to know what all of us can do to stop that from happening. My new book *Incorruptible* is my attempt to explain the invisible forces that shape organizations, and how a handful of companies (like Costco, Patagonia, and Novo Nordisk) have successfully been structured to

resist gravity and thrive for decades -- or even centuries. Along the way, I founded the Long-Term Stock Exchange, co-founded an AI R&D lab called Answer.AI with Jeremy Howard, and helped a number of notable companies with their governance (yes, including Anthropic). I won't pretend I have this all figured out, but I've probably spent more time than is healthy on the "why do good companies go bad" question. Ask me anything! I've helped a lot of people create a lot of amazing companies, but I've also seen so many ways this can go wrong. There's a darkness in our industry that we often don't talk about. I kept watching good companies drift away from the missions they were founded on. Not because anyone woke up one day and decided to be evil, but because the structure they were built on slowly pulled them there. I call that pull "financial gravity." We've all experienced watching a company we love or admire be warped and broken beyond recognition; until it's a husk of its former self, or worse. I wanted to understand why. And I wanted to know what all of us can do to stop that from happening. My new book *Incorruptible* is my attempt to explain the invisible forces that shape organizations, and how a handful of companies (like Costco, Patagonia, and Novo Nordisk) have successfully been structured to resist gravity and thrive for decades -- or even centuries. Along the way, I founded the Long-Term Stock Exchange, co-founded an AI R&D lab called Answer.AI with Jeremy Howard, and helped a number of notable

... [Content truncated. Click below to read more]

READ FULL SOURCE ARTICLE ->



03. Apache Burr: Build reliable AI agents and applications



Apache Burr (Incubating) makes it easy to develop applications that make decisions, from simple chatbots to complex multi-agent systems. Pure Python, no magic. Build anything from chatbots to multi-agent systems with a clean, composable interface. Burr provides the building blocks for reliable, observable, and testable AI-powered applications. Define your application as a set of actions and transitions. No DSL, no YAML just Python functions and decorators. The Burr UI lets you monitor, debug, and trace every step of your application in real time. See state changes as they happen. Automatically persist state to disk, databases, or custom backends. Resume applications from where they left off. Pause execution and wait for human input at any step. Perfect for approval workflows and interactive agents. Run actions in parallel, fan out / fan in, and build complex DAGs. Compose sub-applications for modular design. Replay past runs, unit test individual actions, and validate state transitions. Build confidence in your AI systems. Burr integrates with the tools and frameworks you already use. No lock-in,

no wrappers. See what developers and teams are saying about Burr. After evaluating several other obfuscating LLM frameworks, their elegant yet comprehensive state management solution proved to be the powerful answer to rolling out robots driven by AI decision making. Using Burr is a no-brainer if you want to build a modular AI application. It is so easy to build with and I especially love their UI which makes debugging a piece of cake. And the always ready to help team is the cherry on top. I just came across Burr and I'm like WOW, this seems like you guys predicted this exact need when building this. No weird esoteric concepts just because it's AI. Burr's state management part is really helpful for creating state snapshots and build debugging, replaying and even building evaluation cases around that. I have been using Burr over the past few months, and compared to many agentic LLM platforms out there (e.g. LangChain, CrewAi, AutoGen, Agency Swarm, etc), Burr provides a more robust framework for designing complex behaviors. Moving from LangChain to Burr was a game-changer! It took me just a few hours to get started with Burr, compared to the days and weeks I spent trying to navigate LangChain. I pitched Burr to my teammates, and we pivoted our entire codebase to it. Of course, you can use it [LangChain], but whether it's really production-ready and improves the time from code-to-prod, we've been doing LLM apps for two years, and the answer is no. Honestly, take a look at Burr. Thank me later. After evaluating several other obfuscating LLM ... [Content truncated. Click below to read more]

READ FULL SOURCE ARTICLE ->



04. PgDog is funded and coming to a database near you



The reason DBs like Mongo or Dynamo exist is because Postgres has a scaling problem. If you could make it just work, with 100 TB+ tables and 1M queries per second, we don't think you would use anything else. This is why we are building PgDog. Same old Postgres, just with a proxy in front of it, to make it horizontally scalable. You can deploy PgDog anywhere, including on-prem and in your cloud account: pull our Docker image, change your DATABASE_URL, and make us do the work. PgDog is serving more than 2M queries per second, in production, across dozens of deployments. We sharded over 20 TB that we know about. PgDog is open source and anyone can just deploy it, and they do: we have over 1.4M Docker pulls on GitHub. A new version comes out every week, on Thursdays. Our Discord community is growing. We are there, every day, to answer questions and provide support. PgDog is a small, three-person startup. So, why use our stuff and trust us with your data? We are infrastructure engineers, application engineers and generalists. We built apps on Postgres before it was cool and made it work at massive scale. I ran Postgres

at Instacart, where we scaled the company 5x in April of 2020. The biggest problem we had was making Postgres serve 100,000s of grocery delivery orders per minute. We sharded Postgres on RDS, Aurora and EC2. We fixed the actual problem, using first principles (and a lot of code). The same technology is now available as an open source product. Building PgDog is not a pivot. For us, scaling Postgres has been, and is, the only goal. We built PgDog to run in your cloud, in your colo rack, on-prem, or on your laptop. Wherever you need it, PgDog works, with no dependencies or hidden serverless costs. If you can provide CPUs, our multithreaded code will use them all. Postgres adoption is only going to increase. With \$5.5M from Basis Set, YC, Pioneer Fund and other great investors, we have years of runway, and we are going to make Postgres just work, for everyone, at any scale. P.S. We are building an Enterprise edition of PgDog to make it easier to run in AWS. It comes with SLA-backed support from our team. Give us a call if you want to try it out.

READ FULL SOURCE ARTICLE ->



05. MercedesBenz starts largescale production of electric axial flux motor



This report covers breaking updates regarding 'MercedesBenz starts largescale production of electric axial flux motor'. While detailed public documentation is currently minimal or restricted, industry experts are closely monitoring the situation as it develops. The implications of this update may significantly impact upcoming sector trends and strategies. Please follow the source link below to stay informed on the original publication.

READ FULL SOURCE ARTICLE ->